

Gestion de projet

Vite & Gourmand

Application web de présentation et de commande de menus

TP Développeur Web et Web Mobile
Léna De Bastos

Dépôt GitHub public · Application déployée · Suivi de projet sous Notion

1. Contexte et objectif du document

Ce document explique la manière dont j'ai organisé et piloté le développement de l'application Vite & Gourmand, depuis l'analyse des besoins jusqu'au déploiement. Le projet a été réalisé dans le cadre du TP Développeur Web et Web Mobile, en me plaçant dans la situation d'une développeuse de la société FastDev missionnée par Julie et José pour créer leur site de traiteur.

J'ai mené le projet seule. J'ai donc adapté les méthodes de gestion prévues pour une équipe à une organisation individuelle : garder à tout moment une vision claire de ce qui reste à faire, avancer par petits incréments que je pouvais tester, et documenter au fur et à mesure plutôt qu'à la fin. L'objectif de cette documentation est de rendre visible cette organisation et de justifier les choix que j'ai faits pour tenir le périmètre demandé.

2. Méthode de gestion de projet

J'ai travaillé de façon itérative et incrémentale, en m'inspirant des principes agiles et surtout du Kanban. Le cahier des charges était dense : beaucoup de fonctionnalités, quatre types d'utilisateurs et deux bases de données à faire cohabiter. Découper tout cela en petites tâches livrables m'a permis de mesurer l'avancement réel et d'éviter l'effet tunnel, c'est-à-dire passer des semaines sans rien de fonctionnel à montrer.

N'étant pas en équipe, je n'ai pas mis en place de sprints formels avec cérémonies. J'ai préféré un flux continu : je tirais une tâche, je la terminais et la testais avant d'en commencer une autre, en me limitant à une fonctionnalité en cours à la fois. Pour décider de l'ordre, je me suis appuyée sur la priorisation MoSCoW, qui distingue ce qui est indispensable de ce qui est secondaire.

Priorisation MoSCoW

Catégorie	Signification	Exemples sur le projet
Must have	Indispensable, sans quoi l'application n'a pas de sens	Vue des menus, filtres, commande, comptes et rôles, espace employé
Should have	Important mais contournable au début	Suivi de commande, avis clients validés, réduction 10 %
Could have	Confort, si le temps le permet	Graphique des commandes, calcul du chiffre d'affaires
Won't have	Hors périmètre pour cette version	Paiement en ligne, application mobile native

3. Outil de suivi : Notion

J'ai utilisé Notion comme logiciel de gestion de projet, qui fait partie des outils autorisés par l'énoncé. J'y ai construit un tableau Kanban où chaque carte représente une tâche ou une fonctionnalité. Une carte porte une description tirée du besoin, l'acteur concerné, sa priorité MoSCoW et un lien vers la branche Git correspondante. Faire glisser une carte d'une colonne à l'autre me donnait une image immédiate de l'avancement.

Colonnes du tableau

Colonne	Ce qu'elle contient
À faire	Tâches identifiées et priorisées, pas encore commencées
En cours	La fonctionnalité sur laquelle je travaille (une seule à la fois)
En test	Développement terminé, en cours de vérification sur les différents parcours
Terminé	Testé, validé contre le cahier des charges et fusionné dans la branche de développement

À côté du tableau, j'ai tenu une base de données Notion contenant l'ensemble du backlog. Je pouvais la filtrer par priorité ou par module (menus, commandes, comptes, espace employé, espace administrateur), ce qui m'aidait à décider quoi traiter ensuite et à vérifier que rien n'était oublié.

Lien du tableau Notion : https://app.notion.com/p/3a3933a02d74801c9ec9e526fcda23a6?v=3a3933a02d7480948efb000c61bb9185&source=copy_link

4. Du cahier des charges au backlog

Première étape concrète : relire le cahier des charges et le traduire en user stories, c'est-à-dire des besoins exprimés du point de vue de l'utilisateur, sous la forme « en tant que..., je veux..., afin de... ». Je les ai regroupées par acteur, ce qui correspond aux quatre rôles de l'application et facilite ensuite la gestion des droits.

Acteur	Exemples de user stories	Priorité
Visiteur	Consulter les menus et filtrer ; voir le détail d'un menu ; créer un compte ; contacter l'entreprise	Must
Utilisateur	Commander un menu ; suivre et modifier ses commandes ; déposer un avis après la prestation	Must / Should
Employé	Gérer menus et horaires ; filtrer et faire évoluer le statut des commandes ; valider les avis	Must
Administrateur	Créer et désactiver des comptes employé ; consulter le graphique et le chiffre d'affaires	Must / Could

Ce découpage m'a servi de fil conducteur pour tout le reste. Chaque user story est devenue une ou plusieurs cartes dans Notion, puis une branche Git dédiée au moment du développement.

5. Découpage en phases et planning

L'énoncé prévoit une charge indicative de 70 heures. J'ai réparti le travail en phases qui suivent le cycle de vie d'un projet web, en gardant à l'esprit que ces phases ne sont pas totalement étanches : je revenais parfois sur la conception en cours de développement quand un besoin se précisait.

Phase	Contenu	Charge indicative
Analyse et conception	Analyse des besoins, MCD, choix techniques, maquettes (wireframes et mockups)	~12 h
Environnement et données	Mise en place de l'environnement, base relationnelle et base	~10 h

Phase	Contenu	Charge indicative
	NoSQL, scripts SQL	
Back-end et règles métier	Accès aux données, authentification, rôles, calcul des prix et de la livraison	~18 h
Front-end	Intégration des maquettes, filtres dynamiques sans rechargement, formulaires	~15 h
Sécurité et accessibilité	Mots de passe, contrôle des accès, conformité RGPD et RGAA	~5 h
Tests	Vérification des parcours pour chaque rôle, corrections	~5 h
Déploiement et documentation	Mise en ligne, README, charte graphique, manuel, documentation technique	~5 h

6. Gestion du versionnement (Git)

J'ai suivi le flux de branches demandé dans l'énoncé, qui correspond à une bonne pratique courante en équipe. La branche principale ne reçoit que du code stable et déployable ; tout le travail en cours passe par une branche de développement ; et chaque fonctionnalité vit sur sa propre branche le temps d'être développée et testée.

Branche	Rôle	Règle de fusion
main	Version stable, déployée en production	Reçoit develop uniquement quand tout est testé
develop	Branche d'intégration du travail en cours	Reçoit chaque fonctionnalité après test
feature/...	Une branche par fonctionnalité, issue de develop	Fusionnée dans develop une fois la fonctionnalité validée

Concrètement, pour chaque fonctionnalité je créais une branche à partir de develop (par exemple feature/filtres-menus), je développais, je testais, puis je fusionnais dans develop. Lorsque develop était cohérente et testée dans son ensemble, je la fusionnais dans main. J'ai gardé des commits réguliers et des messages explicites pour pouvoir revenir en arrière facilement. Le dépôt est public sur GitHub, comme demandé, et contient le README expliquant le déploiement en local ainsi que les scripts SQL de création et d'intégration des données.

7. Gestion des risques

Plusieurs points du projet présentaient un risque pour tenir les délais et la qualité. Je les ai identifiés tôt pour prévoir une réponse plutôt que de les subir.

Risque	Mesure prise
Périmètre large, risque de manquer de temps	Priorisation MoSCoW et développement du Must have en premier
Deux bases de données à faire cohabiter (SQL et NoSQL)	Prototypage tôt de la partie MongoDB pour lever le doute
Pénalités si l'application n'est pas en ligne	Déploiement testé tôt et répété, pas gardé pour la fin

Risque	Mesure prise
Sécurité des comptes et des mots de passe	Hachage des mots de passe, règles de complexité, contrôle des accès par rôle
Accessibilité RGAA	Prise en compte dès l'intégration plutôt qu'en correction finale

8. Suivi, tests et validation

Au quotidien, je mettais le tableau Notion à jour à chaque avancée : déplacement des cartes, ajout de sous-tâches quand une fonctionnalité se révélait plus grosse que prévu. Cette habitude m'a évité de perdre le fil et m'a donné une trace de la progression.

Avant de considérer une fonctionnalité comme terminée, je testais manuellement les parcours concernés pour chaque rôle : un visiteur qui filtre puis tente de commander, un utilisateur qui suit et modifie sa commande, un employé qui fait évoluer un statut, un administrateur qui consulte les statistiques. Je reprenais systématiquement le cahier des charges comme liste de contrôle pour vérifier que le comportement correspondait bien à ce qui était attendu, notamment sur les règles de gestion comme la réduction de 10 % ou la facturation de la livraison hors Bordeaux.

9. Bilan et rétrospective

Le découpage en petites tâches et le suivi Kanban ont été les éléments les plus utiles : ils m'ont permis de rester organisée seule sur un projet long et de toujours savoir où j'en étais. Déployer tôt m'a aussi évité la mauvaise surprise d'une mise en ligne qui échoue au dernier moment.

Avec le recul, j'aurais gagné à estimer plus finement certaines tâches dès le départ, car quelques fonctionnalités liées aux commandes se sont révélées plus longues que prévu. Pour un prochain projet, je découperais encore davantage ces user stories et je noterais une estimation de durée sur chaque carte pour mieux anticiper la charge.